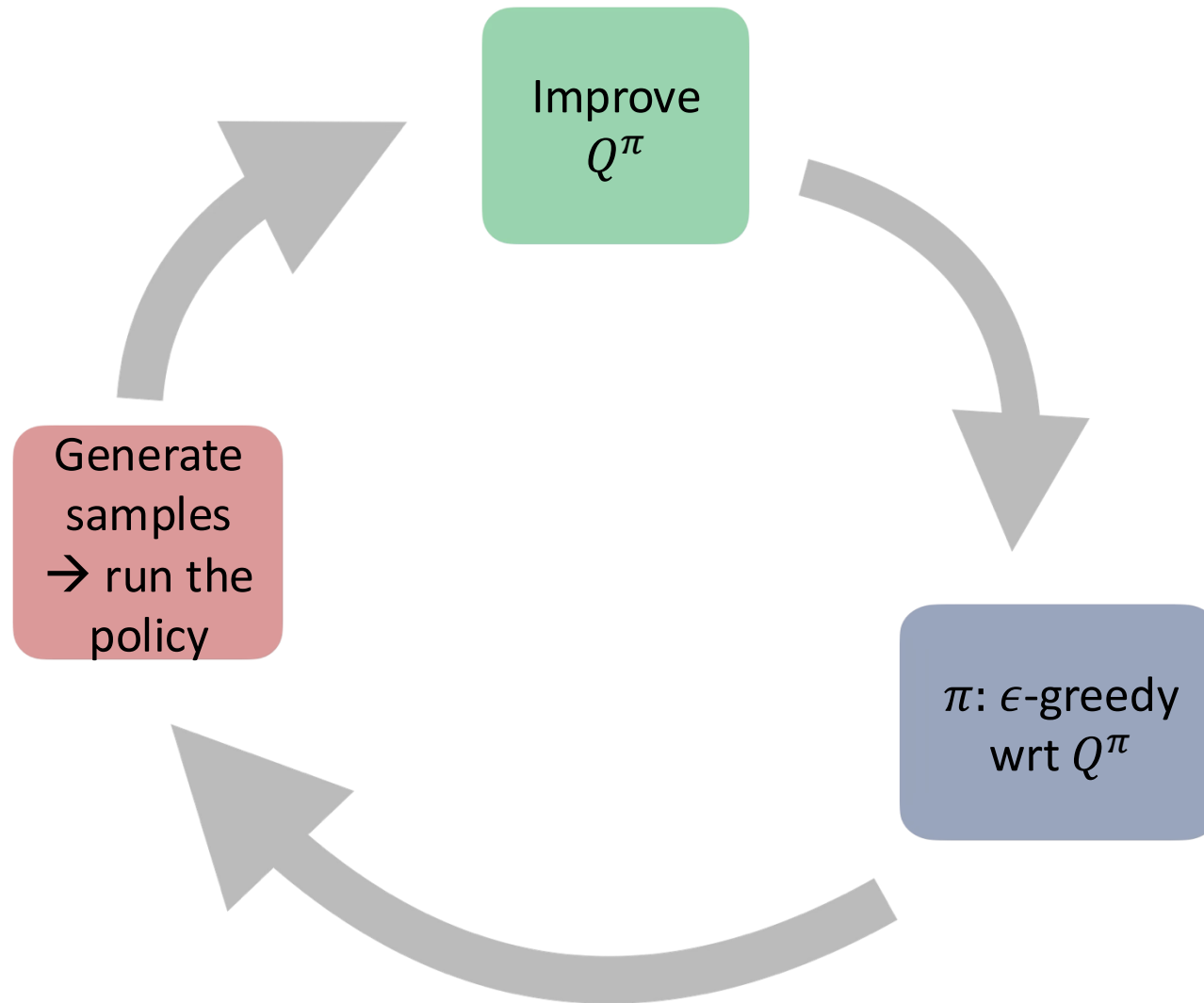


Reinforcement Learning: Off-Policy Learning (ESE 3990)

Instructor:
Antonio Loquercio

Fall 2026
University of Pennsylvania

Model-Free Control (Up to Now)



Learning a Q function with TD(0)

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r(s, a) + \gamma Q(s', a') - Q(s, a))$$

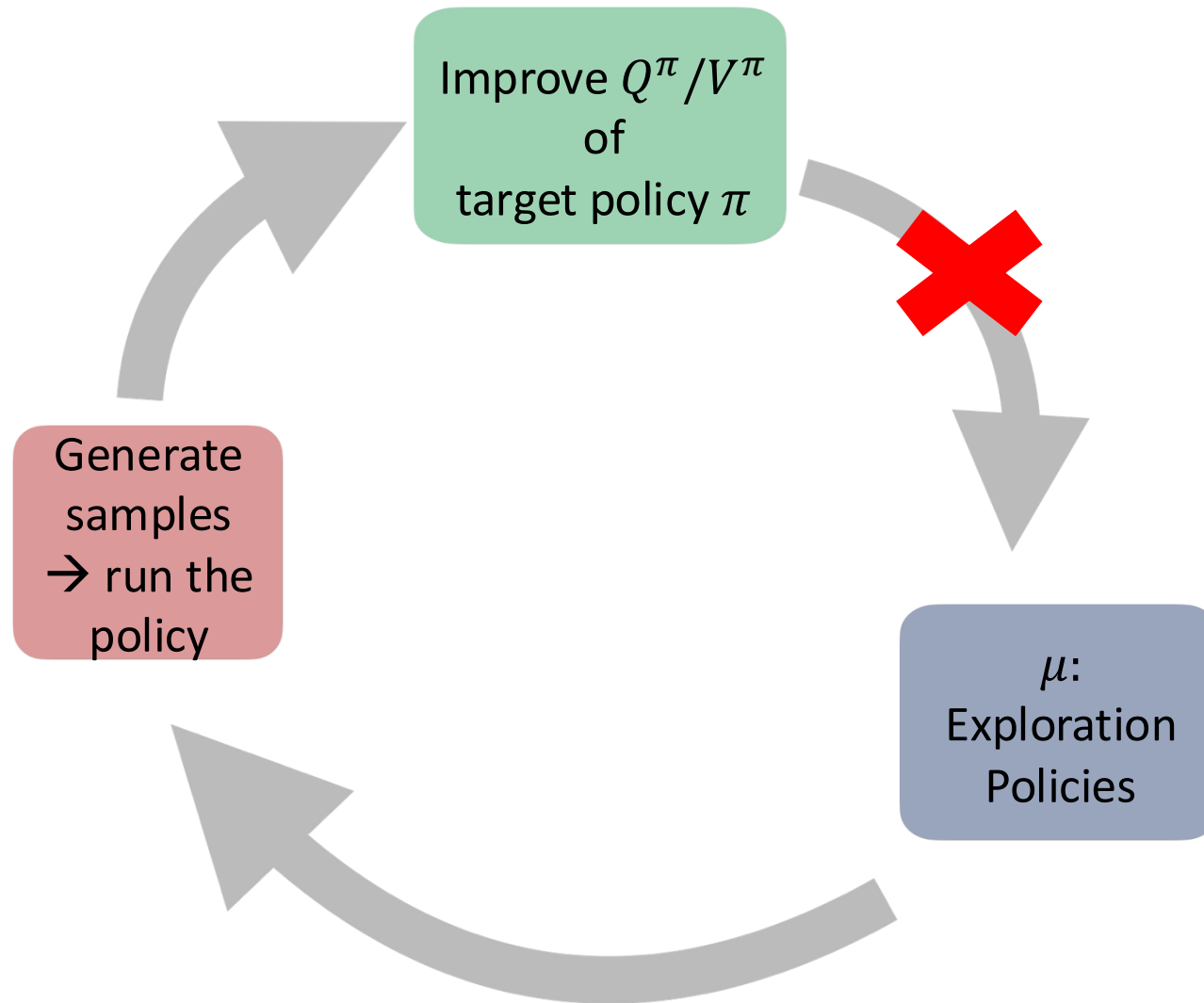
- How to get a' ?
- Option 1 (SARSA): Sample from the current policy
$$a' = \pi(\cdot | s')$$
- Option 2 (Q-Learning): Sample the **greedy** action at the next step
$$a' = \operatorname{argmax}_a Q(s', a)$$

Q-learning Control Algorithm (Watkin, 89)

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

- In contrast to Sarsa, this is **not** attempting to learn Q^π , but Q^*
 - Because the next action is not selected according to π (which is ϵ -greedy)
- Example of an *off-policy* algorithm

Off-Policy Learning



Off-Policy Learning

- Evaluate **target policy** $\pi(a|s)$ to compute $V^\pi(s)$ or $Q^\pi(s, a)$
- While following the **behavior policy** $\mu(a|s)$
$$\{s_1, a_1, r_2, \dots, s_T\} \sim \mu$$
- Why is this important?
 - Learn about *optimal* policy while following *exploratory* policy.
 - Learn about *multiple* policies (rewards) while following one policy.
 - Learn from *observing* other agents' state-action trajectories (e.g., humans).
 - Sample efficiency (more on this later)

Off-Policy Model-Free Prediction of V^π

- *Fixed* target policy π
- *Fixed* exploration policy μ
- *Goal*: Learn $V^\pi(s)$
- We will later generalize to Q^π

Importance Sampling

- Estimate the expectation of a different distribution

$$\begin{aligned}\mathbb{E}_{X \sim P}[f(X)] &= \sum P(X)f(X) \\ &= \sum Q(X) \frac{P(X)}{Q(X)} f(X) \\ &= \mathbb{E}_{X \sim Q} \left[\frac{P(X)}{Q(X)} f(X) \right]\end{aligned}$$

Importance Sampling: Example

- Assume you roll 5 dice randomly
- What is the probability of getting a sum of 30?
 - $p(\text{outcome} = 6)^5 = 0.0128\%$
- If you try to estimate this event by sampling, you will likely hit a sum of 30 once every 10'000 runs.
- You might need millions of runs to get a good estimate of $p(\text{sum} = 30)$

Importance Sampling: Example

- To be faster, you decide to use biased dice that roll 6s more often.
- Now, you get plenty of 30-sum events quickly.
- But you can't use the samples as is:
 - Your estimate will be inflated by the biased dice

Importance Sampling: Example

- Importance sampling tells us how to fix the biased estimate
- Reweight each sample based on how much you cheated:

$$W = \frac{\textit{True Probability of outcome}}{\textit{Biased Probability of outcome}}$$

Importance Sampling: Example

- Say we cheated with $p_\mu(\text{outcome} = 6) = \frac{5}{6}$; $p_\mu(\text{outcome} \neq 6) = \frac{1}{30}$
- Then the weight should be

$$W = \frac{\left(\frac{1}{6}\right)^5}{\left(\frac{5}{6}\right)^5} = \frac{1}{5^5} = 0.00032$$

- So, we should discount the frequency of observing a sum of 30 by a very small number.
- But at least it is a number bigger than zero! (useful for rare events)

Importance Sampling: Example

- Say we cheated with $p_\mu(\text{outcome} = 6) = \frac{5}{6}$; $p_\mu(\text{outcome} \neq 6) = \frac{1}{30}$
- What about estimating $p(\text{sum} = 5)$ with this biased dice?

$$W = \frac{\left(\frac{1}{6}\right)^5}{\left(\frac{1}{30}\right)^5} = \frac{30^5}{6^5} = 5^5 = 3'125$$

- We should upweight the frequency by a very large number
- Can create a lot of variance!

Importance Sampling: Example

- Summary:
 - The more the dice is biased towards the outcome I am interested in, the more accurate the estimate it.
 - The further away from it, the bigger the variance.
 - Over infinite-amount of samples (i.e. throws) it won't matter how I design the dice.
- The same can be said for exploration policies in off-policy learning.

Evaluating V^π with Importance Sampling

$$V^\pi(s_t) = \mathbb{E}_{\tau=(a_t, s_{t+1}, a_{t+1}, \dots, s_T) \sim \pi} \left[\sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{t'}, a_{t'}) \right]$$

$$V^{\pi \setminus \mu}(s_t) = \mathbb{E}_{\tau=(a_t, s_{t+1}, a_{t+1}, \dots, s_T) \sim \mu} \left[\frac{P(\tau; \pi)}{Q(\tau; \mu)} \sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{t'}, a_{t'}) \right]$$

Evaluating V^π with Importance Sampling

$$P(\tau; \pi) = \pi(a_t|s_t)p(s_{t+1}|a_t, s_t)\pi(a_{t+1}|s_{t+1}) \cdots$$

$$Q(\tau; \mu) = \mu(a_t|s_t)p(s_{t+1}|a_t, s_t)\mu(a_{t+1}|s_{t+1}) \cdots$$

The correction weight of each return is the ratio of action probabilities:

$$W = \frac{P(\tau; \pi)}{Q(\tau; \mu)} = \frac{\pi(a_t|s_t)}{\mu(a_t|s_t)} \frac{\pi(a_{t+1}|s_{t+1})}{\mu(a_{t+1}|s_{t+1})} \cdots$$

- Since a policy cannot affect the MDP transition probabilities.
 - Can you think of a case where this is not a good assumption?

Evaluating V^π with Off-Policy Monte-Carlo

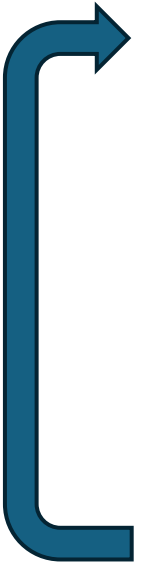
$$\begin{aligned} V^{\pi \setminus \mu}(s_t) &= \mathbb{E}_{\tau=(a_t, s_{t+1}, a_{t+1}, \dots, s_T) \sim \mu} \left[\frac{P(\tau; \pi)}{Q(\tau; \mu)} \sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{t'}, a_{t'}) \right] \\ &= \mathbb{E}_{\tau=(a_t, s_{t+1}, a_{t+1}, \dots, s_T) \sim \mu} [\rho_{t:T-1} G_t] \end{aligned}$$

Where:

$$\rho_{t:T-1} = \frac{\pi(a_t | s_t)}{\mu(a_t | s_t)} \frac{\pi(a_{t+1} | s_{t+1})}{\mu(a_{t+1} | s_{t+1})} \cdots \frac{\pi(a_{T-1} | s_{T-1})}{\mu(a_{T-1} | s_{T-1})}$$

- *Intuition.* Weight return G_t according to “similarity” between policies

Importance Sampling for Off-Policy Monte-Carlo

- 
- Collect samples by running μ in the environment

- $$G_t^{\pi \setminus \mu} = \frac{\pi(a_t|s_t)}{\mu(a_t|s_t)} \frac{\pi(a_{t+1}|s_{t+1})}{\mu(a_{t+1}|s_{t+1})} \cdots \frac{\pi(a_{T-1}|s_{T-1})}{\mu(a_{T-1}|s_{T-1})} G_t$$

- Update the value of the target policy π towards *the corrected* return

$$V(s_t) \leftarrow V(s_t) + \alpha \left(G_t^{\pi \setminus \mu} - V(s_t) \right)$$

Importance Sampling for Off-Policy Monte-Carlo

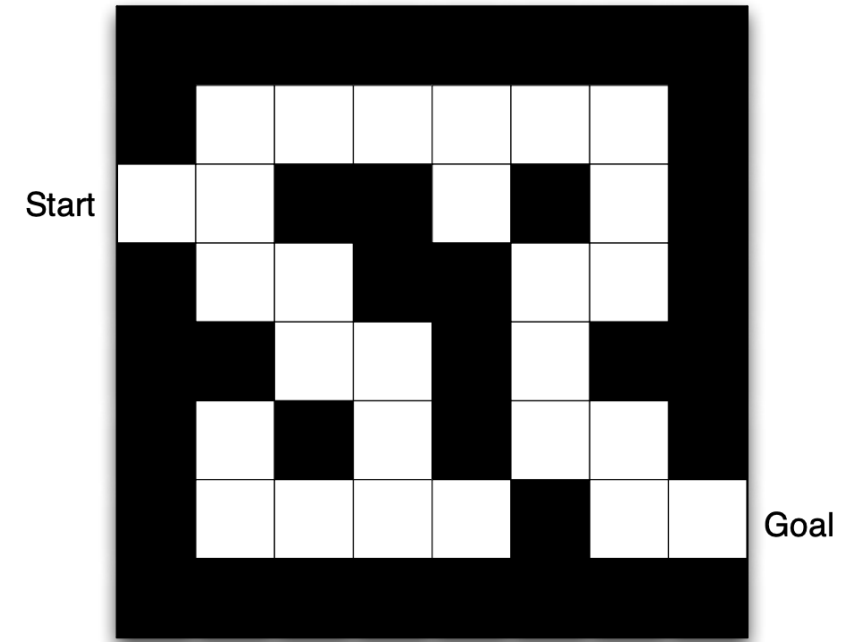
- $G_t^{\pi \setminus \mu} = \frac{\pi(a_t|s_t)}{\mu(a_t|s_t)} \frac{\pi(a_{t+1}|s_{t+1})}{\mu(a_{t+1}|s_{t+1})} \cdots \frac{\pi(a_{T-1}|s_{T-1})}{\mu(a_{T-1}|s_{T-1})} G_t$
- *Issue:*
 - Cannot use if $\mu > 0$ and $\pi = 0$

Off-Policy Learning: Core Assumption

- Every action taken by the target π is also taken (at least occasionally) by the exploration policy μ .
 - $\pi(a|s) > 0 \rightarrow \mu(a|s) > 0$
- This is called *coverage*
- It follows from coverage that $\mu(a|s)$ *must be* stochastic
 - Even if $\pi(a|s)$ is deterministic
- Is there a case when $\mu(a|s)$ can be deterministic and still have coverage?

Off-Policy Monte-Carlo In Maze

- π : optimal policy
- μ : random policy
- $R(goal) = 100$, zero otherwise.



- Suppose you observe one lucky episode:

$$G_t^{\pi \setminus \mu} = \frac{\pi(a_t|s_t) \pi(a_{t+1}|s_{t+1})}{\mu(a_t|s_t) \mu(a_{t+1}|s_{t+1})} \cdots \frac{\pi(a_T|s_T)}{\mu(a_T|s_T)} 100$$

- Is the weighting factor $\rho_{t:T-1}$ small or large?

Importance Sampling for Off-Policy Monte-Carlo

- $G_t^{\pi \setminus \mu} = \frac{\pi(a_t|s_t)}{\mu(a_t|s_t)} \frac{\pi(a_{t+1}|s_{t+1})}{\mu(a_{t+1}|s_{t+1})} \cdots \frac{\pi(a_{T-1}|s_{T-1})}{\mu(a_{T-1}|s_{T-1})} G_t$
- *Issue:*
 - Cannot use if μ is zero and π is non-zero
 - The correction weight can drastically *increase* variance

Reducing Variance: Weighted Importance Sampling

- Suppose we have a set of returns all starting from s_t : $G_t^1, G_t^2, \dots, G_t^K$
- In MC with ordinary importance sampling:

$$V(s) = \frac{1}{K} \sum_k \rho_{t:T-1}^k G_t^k$$

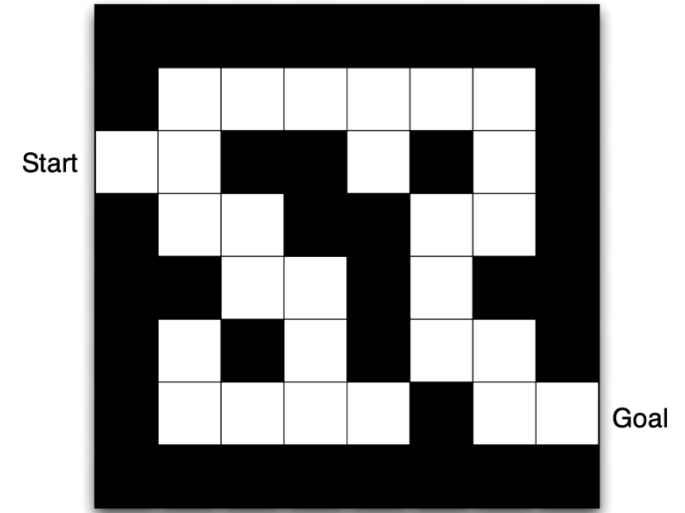
- In **weighted** importance sampling

$$V(s) = \frac{1}{\sum_k \rho_{t:T-1}^k} \sum_k \rho_{t:T-1}^k G_t^k$$

Reducing Variance: Weighted Importance Sampling

Weighted importance sampling

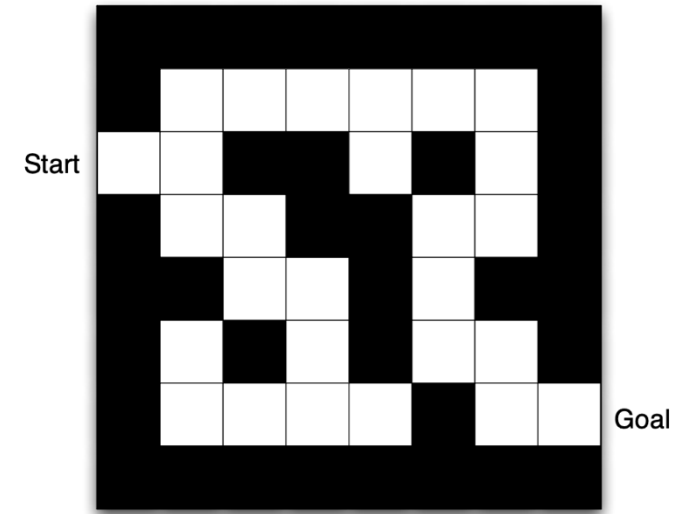
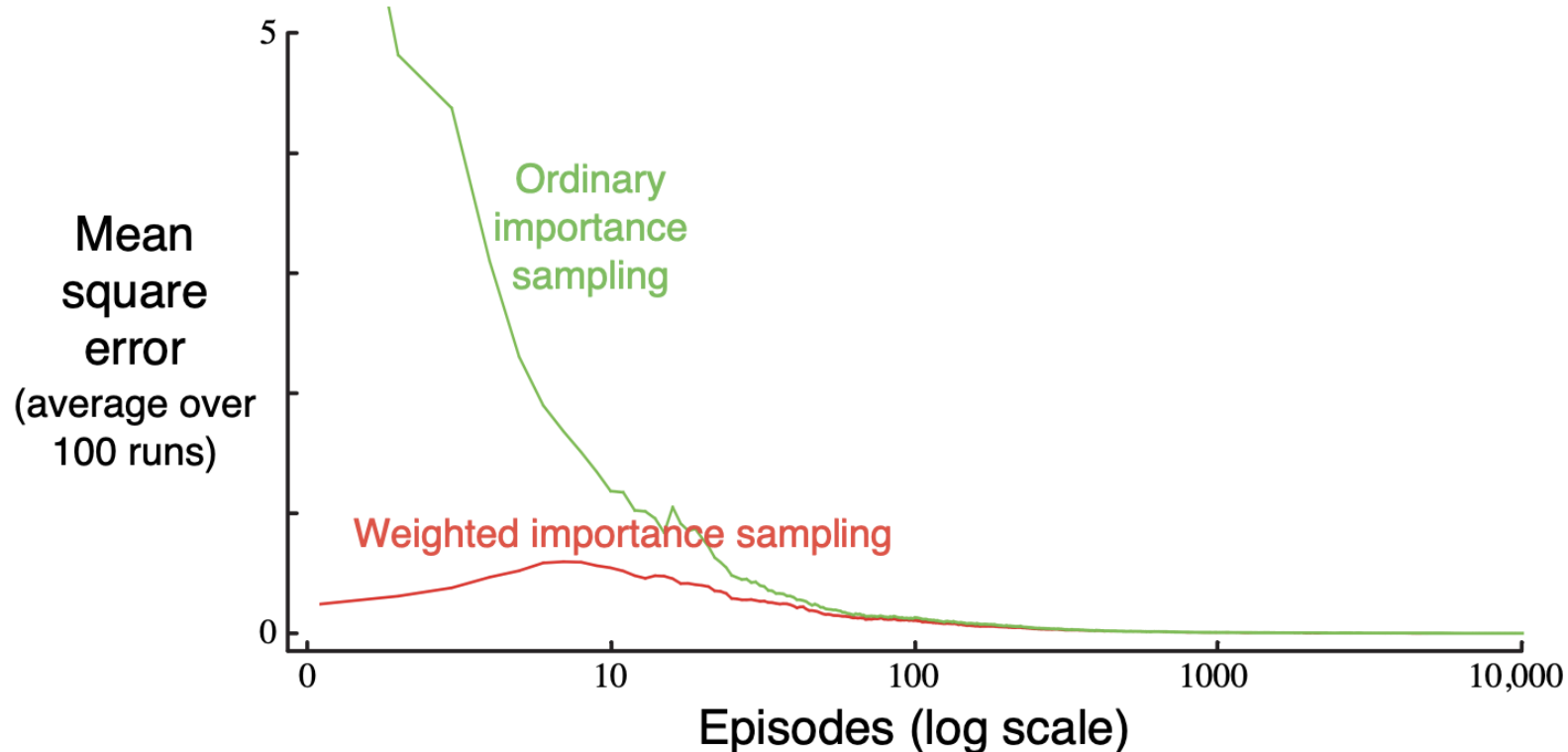
$$V(s) = \frac{1}{\sum_k \rho_{t:T-1}^k} \sum_k \rho_{t:T-1}^k G_t^k$$



Intuition

- If in the first few episodes you observe a very large return, the denominator scales it back.
- Reduces variance, but adds bias in the early episodes
 - In the very first episode, there is no correction weight!

Reducing Variance: Weighted Importance Sampling



Reducing Variance: Weighted Importance Sampling

- Bias decreases to zero as $K \rightarrow \infty$

- **Proof Sketch:**

$$\mathbb{E}_{a \sim \mu} \left[\frac{\pi(a|s)}{\mu(a|s)} \right] = \sum_a \frac{\mu(a|s)\pi(a|s)}{\mu(a|s)} = \sum_a \pi(a|s) = 1$$

- Using induction, it is possible to show that:

$$\mathbb{E}_{(a_t, s_t, a_{t+1}, \dots, a_{T-1}, s_{T-1}) \sim \mu} [\rho_{t:T-1}] = 1$$

- Therefore:

$$\bullet V(s) = \frac{1}{\sum_k \rho_{t:T-1}^k} \sum_k \rho_{t:T-1}^k G_t^k = \frac{1}{K} \sum_k \rho_{t:T-1}^k G_t^k \text{ as } k \rightarrow \infty$$

Reducing Variance: Per-decision Importance Sampling

- Weighted importance sampling can only be used for (small) discrete state-action spaces.
- We need another approach that works even without such assumptions.
- Per-decision importance sampling!

Reducing Variance: Per-decision Importance Sampling

- Per-decision importance sampling:

$$\begin{aligned}\rho_{t:T-1}G_t &= \rho_{t:T-1}(r_t + \gamma r_{t+1} + \dots + \gamma^{T-t-1}R_T) \\ &= \rho_{t:T-1}r_t + \gamma\rho_{t:T-1}r_{t+1} + \dots\end{aligned}$$

- For each element in the sum, one can show that

$$\mathbb{E}_{(a_t, s_t, \dots, a_{T-1}, s_{T-1}) \sim \mu} [\rho_{t:T-1} r_{t'}] = \mathbb{E}_{(a_t, s_t, \dots, a_{T-1}, s_{T-1}) \sim \mu} [\rho_{t:t'} r_{t'}]$$

- Similar proof as before.
- **Intuition.** Causality: Future actions do not affect current reward.

Reducing Variance: Per-decision Importance Sampling

$$\begin{aligned} V^\pi(s_t) &= \mathbb{E}_{(a_t, \dots, a_{T-1}) \sim \mu} [\rho_{t:T-1} G_t] \\ &= \sum_{t'=t}^{T-1} \gamma^{t+t'-1} \mathbb{E}_{(a_t, \dots, a_{t'}) \sim \mu} [\rho_{t:t'} r_{t'}] \end{aligned}$$

- Therefore, I can use the following weight in off-policy MC:

$$G_t^{\pi \setminus \mu} = \rho_{t:t} r_t + \gamma \rho_{t:t+1} r_{t+1} + \dots + \gamma^{T-t-1} \rho_{t:T-1} r_T$$

- Why does this have less variance than the ordinary IS return?

$$G_t^{\pi \setminus \mu} = \rho_{t:T-1} (r_t + \gamma r_{t+1} + \dots + \gamma^{T-t-1} R_T)$$

Reducing Variance with Off-Policy TD(0)

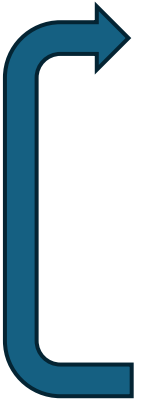
- TD(0) uses data from a single interaction step.
- **Refresher.** Using the bellman expectation equation:

$$V^\pi(s_t) = \mathbb{E}_{(a_t, s_{t+1}) \sim \pi(\cdot | s_t)} [r(a_t, s_t) + \gamma V^\pi(s_{t+1})]$$

- Using the importance sampling idea:

$$V^\pi(s_t) = \mathbb{E}_{(a_t, s_{t+1}) \sim \mu(\cdot | s_t)} \left[\frac{\pi(a_t | s_t)}{\mu(a_t | s_t)} (r(a_t, s_t) + \gamma V^\pi(s_{t+1})) \right]$$

Importance Sampling for Off-Policy TD



- Run μ in the environment for a single step, observe (s_t, r_t, a_t, s_{t+1})

- Update the value of the target policy π

$$V(s_t) \leftarrow V(s_t) + \alpha \left(\frac{\pi(a_t|s_t)}{\mu(a_t|s_t)} (r_t + \gamma V(s_{t+1})) - V(s_t) \right)$$

- Exercise at home: Find the equivalent formula for TD(n).

Summary: Up to Now

- We have seen several ways to find V^π given experience from μ
- All methods are based on the idea of weight the returns according to the “similarity” between the policies.
- Off-policy MC
 - W/ Ordinary Importance Sampling
 - W/ Weighted Importance Sampling
 - W/ Per-Decision Importance Sampling
- Off-Policy TD(0)
 - Possible to extend to TD(n) and TD(λ)
 - Can further reduce variance with the techniques above

Learning Q^π

- We will now see how to adapt these techniques when learning a Q function:
- Off-policy MC
 - W/ Ordinary Importance Sampling
 - W/ Weighted Importance Sampling
 - W/ Per-Decision Importance Sampling
- Off-Policy TD(0)
 - Possible to extend to TD(n) and TD(λ)

Evaluating Q^π with Off-Policy Monte-Carlo

- **Refresher.** The Q function is defined as:

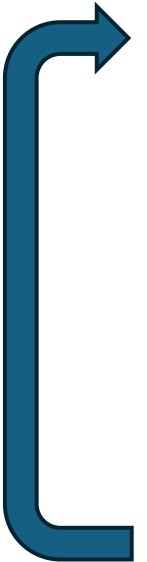
$$Q^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1}, s_{t+1}, \dots, a_{T-1}, s_{T-1} \sim \pi} [G_{t+1}]$$

- Using the importance sampling idea:

$$Q^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1}, s_{t+1}, \dots, a_{T-1}, s_{T-1} \sim \mu} [\rho_{t+1:T-1} G_{t+1}]$$

- When learning the Q function, we start the policy ratio at $t + 1$!

Evaluating Q^π with Off-Policy Monte-Carlo

- 
- Collect samples by running μ in the environment

- $G_{t+1}^{\pi \setminus \mu} = \frac{\pi(a_{t+1}|s_{t+1})}{\mu(a_{t+1}|s_{t+1})} \cdots \frac{\pi(a_{T-1}|s_{T-1})}{\mu(a_{T-1}|s_{T-1})} G_t$

- Update the Q of the target policy π towards *the corrected* return

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r(s_t, a_t) + \gamma G_{t+1}^{\pi \setminus \mu} - Q(s_t, a_t) \right)$$

Evaluating Q^π with Off-Policy TD(0)

- **Refresher.** The Q function is defined as:

- $$Q^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{(s_{t+1}|a_t)}[V^\pi(s_{t+1})]$$
$$= r(s_t, a_t) + \gamma \mathbb{E}_{(s_{t+1}|a_t)}[\mathbb{E}_{(a_{t+1}) \sim \pi(s_{t+1})}[Q^\pi(s_{t+1}, a_{t+1})]]$$

- Using the importance sampling idea:

$$Q^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{(a_{t+1}) \sim \mu(s_{t+1})} \left[\frac{\pi(s_{t+1})}{\mu(s_{t+1})} Q^\pi(s_{t+1}, a_{t+1}) \right]$$

- We can do even better!

Evaluating Q^π with Off-Policy TD(0)

- **Refresher.** The Q function is defined as:
 - $Q^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{(s_{t+1}|a_t, s_t)} [\mathbb{E}_{(a_{t+1}) \sim \pi(s_{t+1})} [Q^\pi(s_{t+1}, a_{t+1})]]$
 - $p(s_{t+1}|a_t, s_t)$ is policy independent.
- $V^\pi(s_t) = \mathbb{E}_{(a_t, s_{t+1}) \sim \pi(\cdot|s_t)} [r(a_t, s_t) + \gamma V^\pi(s_{t+1})]$
- For V^π , the next state depends on the action a_t *selected by the policy π .*

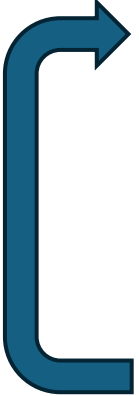
Evaluating Q^π with Off-Policy TD(0)

- So potentially, using importance sampling, I could write:

$$Q^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{(s_{t+1}|a_t, s_t)} \left[\mathbb{E}_{a_{t+1} \sim \mu(s_{t+1})} \left[\frac{\pi(s_{t+1})}{\mu(s_{t+1})} Q^\pi(s_{t+1}, a_{t+1}) \right] \right]$$

- Since s_{t+1} does not depend on π or μ , we can skip importance sampling altogether in the second expected value.
- I can just sample a_{t+1} according to π (which I know)!
- Therefore, **no** importance sampling is required in off-policy TD(0) for Q^π .

Importance Sampling for Off-Policy TD

- 
- Run μ in the environment for a single step, observe (s_t, r_t, a_t, s_{t+1})
 - Update Q of the target policy π
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_t + \gamma Q(s_{t+1}, a') - Q(s_t, a_t))$$
 - With $a' \sim \pi(s_{t+1})$, sampled from the current policy
 - Exercise at home: Find the equivalent formula for TD(n).

TD(0) Q^π : On-Policy vs Off-Policy

- *Idea*. Just change the successor action a_{t+1} .
 - *On-policy*:
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)),$$
 - with a_{t+1} sampled from $\mu(\cdot | s_{t+1})$
 - *Off-policy*:
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_{t+1} + \gamma Q(s_{t+1}, a') - Q(s_t, a_t)),$$
 - with a' sampled from $\pi(\cdot | s_{t+1})$
- **General Insight**: In off-policy RL, IS required when computing expectations over states dependent on the exploration policy.

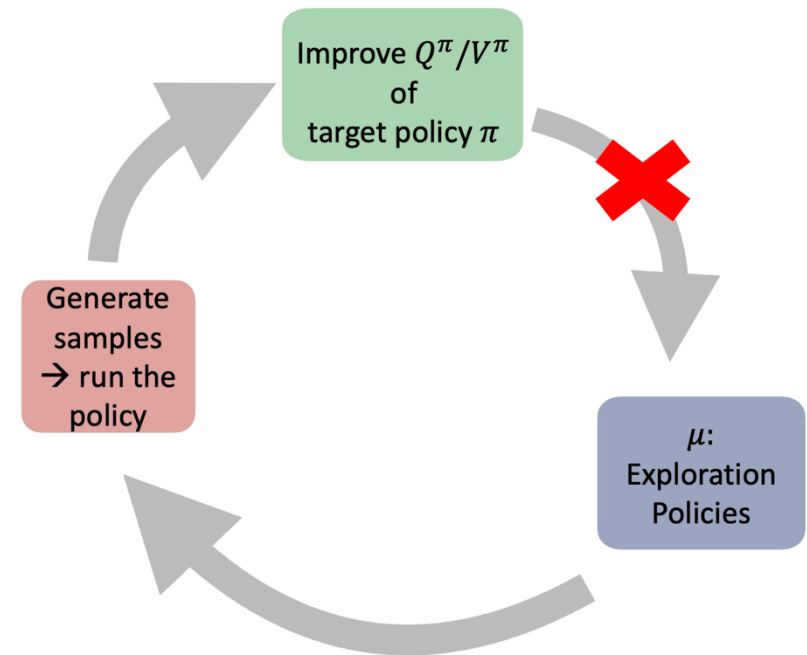
Remember?

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_{t+1} + \gamma Q(s_{t+1}, a') - Q(s_t, a_t))$$

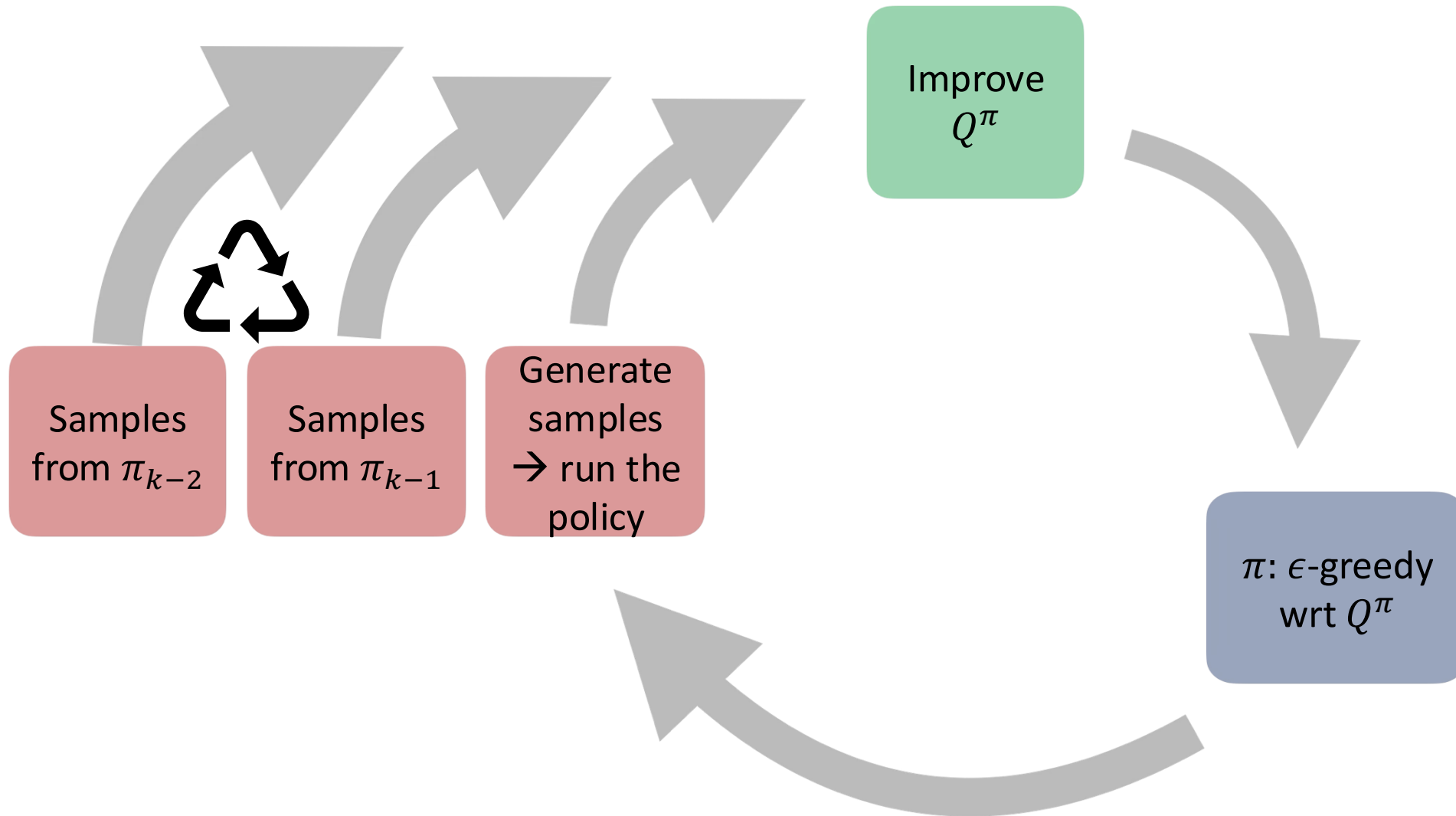
- How to get a' ?
- Option 1 (SARSA): Sample from the current policy
$$a' = \pi(\cdot | s')$$
- Option 2 (Q-Learning): Sample the **greedy** action at the next step
$$a' = \operatorname{argmax}_a Q(s', a)$$

Summary: Model-Free Off-Policy Prediction

- All methods are based on the idea of weight the returns according to the “similarity” between the policies.
- There are a few differences between the definition of “similarity” when learning V^π and Q^π :
 - Start the policy ratio at $t+1$ when using MC
 - No important sampling for TD(0).



Model-Free Off-Policy Control



Off-Policy Model-Free Control

- Algorithm summary:
 - Step 0: Init with a random policy π
 - Step 1: Use a *single* episode to learn $Q^\pi(s, a)$.
 - Step 2: $\pi \leftarrow \epsilon\text{-greedy}(Q^\pi(s, a))$
 - Step 3: Collect one episode with π , use all prior episodes (latest included) to learn $Q^\pi(s, a)$.
 - ...
- Effectively, we are using prior policies as “exploration” policies μ .

Q-Learning

- Extreme case of the previous algorithm
- Algorithm summary:
 - Step 0: Init with a random policy π
 - Step 1: Use a *single* episode to estimate ~~$Q^\pi(s, a)$~~ . $Q^*(s, a)$
 - Step 2: $\pi \leftarrow \epsilon$ -greedy(~~$Q^\pi(s, a)$~~ . $Q^*(s, a)$)
 - Step 3: Collect one episode with π , use all prior episodes (latest included) to estimate ~~$Q^\pi(s, a)$~~ . $Q^*(s, a)$
 - ...

Q-Learning: How can I sample from π^* ?

- The Q-learning target then simplifies to:

$$\begin{aligned} & r_{t+1} + \gamma Q(s_{t+1}, a') \\ &= r_{t+1} + \gamma Q\left(s_{t+1}, \arg \max_{a'} Q(s_{t+1}, a')\right) \\ &= r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') \end{aligned}$$

- Assume that the optimal policy is greedy with respect to current Q .
- Possible to prove convergence with contraction-theory.

Maximization Bias

- Assume the true $Q^*(s, a) = 0, \forall a$
- Assume our current estimate of $Q_k(s, a) = \mathcal{N}(0, \sigma)$, where σ is the RMSE of our current estimate.
- $\max_a Q^*(s, a) = 0$
- $\max_a Q_k(s, a) = \beta > 0$

Maximization Bias: Q-Learning Update

Q-learning update

$$Q_{k+1}(s, a) = Q_k(s, a) + \alpha \left(r(s, a) + \gamma \max_{a' \in A} Q(s', a') - Q_k(s, a) \right)$$

- If using $\max_a Q^*(s, a) = 0$

$$Q_{k+1}(s, a) = Q_k(s, a) + \alpha(0 - Q_k(s, a)) < |Q_k(s, a)|$$

- If using $\max_a Q_k(s, a) = \beta > 0$

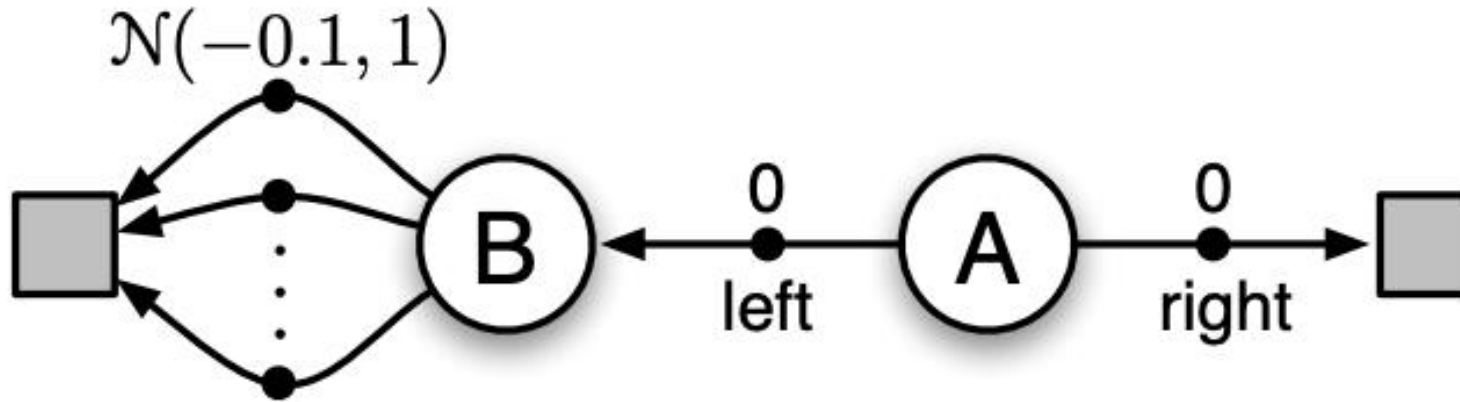
$$Q_{k+1}(s, a) = Q_k(s, a) + \alpha(\gamma\beta - Q_k(s, a)) \geq Q_k(s, a)$$

- The estimated q values are growing!

Maximization Bias

- Even Q_k is a good estimate of Q^*
 - In our case, it was unbiased in expectation
- The maximum over estimated values is generally *larger* than an estimate of the maximum.
- This is called maximization bias
- Not only a problem for Q-learning but also for SARSA (since we do ϵ -greedy action selection).

Example



- $Q^*(A, right) = 0$
- $Q^*(A, left) = V(B) = -0.1$
- When doing Q learning, which action do you think will be most likely sampled?

Maximization Bias: Double Q-Learning

- The problem is that we're estimating the maximum of the true value as the maximum of value estimates.
- This stems from the fact that we're using the same data to:
 - Estimate the maximum action $\widehat{a}^* = \arg \max Q_k(s, a)$
 - Estimate the value of the maximum action $\widehat{Q}^*(s, a) = Q_k(s, \widehat{a}^*)$
- **Idea.** What if we use two different data sources for these goals?
 - Estimate the maximum action $\widehat{a}^* = \arg \max Q_k^1(s, a)$
 - Estimate the value of the maximum action $\widehat{Q}^*(s, a) = Q_k^2(s, \widehat{a}^*)$

Double Q-Learning (Basic)

- Maintain two estimates of Q functions, Q_k^1 and Q_k^2
- At every update step, flip a coin:
- If tails,
 - $Q_{k+1}^1(s, a) = Q_k^1(s, a) + \alpha \left(r(s, a) + \gamma Q_k^2 \left(s', \arg \max_{a'} Q_k^1(s', a') \right) - Q_k^1(s, a) \right)$
 - $Q_{k+1}^2 = Q_k^2$
- If heads,
 - $Q_{k+1}^2(s, a) = Q_k^2(s, a) + \alpha \left(r(s, a) + \gamma Q_k^1 \left(s', \arg \max_{a'} Q_k^2(s', a') \right) - Q_k^2(s, a) \right)$
 - $Q_{k+1}^1 = Q_k^1$

Double-Q Learning vs Q-Learning

